

INTRODUZIONE

Che cosa è React

Definizione. React è una **libreria JavaScript** (non un framework) per costruire interfacce utente. La sua idea centrale è che l'interfaccia viene spezzata in **componenti**: pezzi di UI autonomi, riutilizzabili, che si compongono fra loro come mattoncini.

La differenza fondamentale rispetto al JavaScript "vanilla" è il modello di aggiornamento. In JavaScript classico tu dici al browser *come* modificare la pagina, passo per passo:

```
// JavaScript classico: descrivo i PASSI  
  
const titolo = document.getElementById("titolo");  
titolo.textContent = "Nuovo titolo";  
titolo.classList.add("evidenziato");
```

In React tu dichiari *come deve apparire* l'interfaccia dato un certo stato, e React si occupa di aggiornare il DOM:

```
// React: descrivo il RISULTATO  
  
function Titolo({ testo, evidenziato }) {  
  return <h1 className={evidenziato ? "evidenziato" : ""}>{testo}</h1>;  
}
```

Questo si chiama approccio **dichiarativo** contro **imperativo**.

Perché è veloce. React mantiene in memoria una copia leggera della struttura della pagina, il **Virtual DOM**. Quando qualcosa cambia, confronta la versione nuova con la vecchia (processo detto *reconciliation*) e tocca nel DOM reale solo le parti effettivamente cambiate. Il DOM reale è lento da manipolare; il confronto in memoria no.

Esempio concreto del beneficio. Hai una lista di 500 prodotti e cambi il prezzo del prodotto numero 237. Senza React rischi di ridisegnare l'intera lista. React confronta, capisce che è cambiato un solo nodo di testo, e aggiorna solo quello.

Competenze minime

Il corso è esplicito: React non si studia senza basi. I prerequisiti indicati sono tre, con tanto di video di ripasso linkati nelle trascrizioni:

- **HTML** (1h 33m di ripasso) — struttura dei documenti, tag semantici, attributi.
- **CSS** (5h 27m) — selettori, box model, flexbox, grid, media query.
- **JavaScript** (6h 12m) — e qui sta la parte critica.

Le parti di JavaScript che servono davvero, perché le ritroverai in ogni riga di React:

Arrow functions. Sintassi compatta per le funzioni. Ogni componente moderno è scritto così.

```
// funzione classica
function somma(a, b) { return a + b; }

// arrow function
const somma = (a, b) => a + b;
```

Destrutturazione. Estrarre valori da oggetti e array con una sintassi breve.

```
const utente = { nome: "Dario", eta: 30 };
const { nome, eta } = utente; // nome = "Dario", eta = 30
```

Spread operator (. . .). Espandere il contenuto di un oggetto o array.

```
const base = { nome: "Dario", eta: 30 };
const completo = { ...base, citta: "Barletta" };
```

Metodi degli array, soprattutto `map` e `filter`:

```
const numeri = [1, 2, 3];
const doppi = numeri.map(n => n * 2);    // [2, 4, 6]
const pari = numeri.filter(n => n % 2 === 0); // [2]
```

Template literals.

```
const nome = "Dario";
console.log(`Ciao ${nome}, benvenuto`);
```

Promise e `async/await`, perché il recupero dati dalle API si fa così:

```
const risposta = await fetch("https://api.esempio.com/dati");
const dati = await risposta.json();
```

Se questi cinque concetti non sono automatici, React sembrerà incomprensibile — non perché React sia difficile, ma perché stai leggendo JavaScript che non conosci.

Dev Environment (l'ambiente di sviluppo)

Definizione. L'insieme di programmi installati sul tuo computer che ti permettono di scrivere, eseguire e testare codice.

Gli elementi richiesti dal corso:

Node.js. È un runtime che permette di eseguire JavaScript fuori dal browser. Serve perché gli strumenti di sviluppo React (compilatore, dev server, gestore pacchetti) sono scritti in JavaScript e girano sul tuo computer, non nel browser. Verifica dell'installazione:

```
node -v # es. v20.11.0
```

```
npm -v    # es. 10.2.4
```

npm (Node Package Manager). Si installa insieme a Node. È il programma che scarica e gestisce le librerie di terze parti.

```
npm install axios    # installa la libreria axios nel progetto
```

```
npm install          # installa tutto ciò che è elencato in package.json
```

npx. Qui la trascrizione dice "EMP X" e il corso lo spiega bene: npx è simile a npm, ma mentre `npm install` scarica e installa una libreria nel progetto, **npx esegue direttamente uno script** senza installarlo in modo permanente. Serve esattamente per comandi usa-e-getta come la creazione di un progetto.

Visual Studio Code. L'editor consigliato. Le estensioni citate nel corso:

- **ES7+ React/Redux snippets** — quella che genera i componenti con la scorciatoia `rfc` (ne riparlamo fra poco).
- **Prettier** — formatta automaticamente il codice al salvataggio.
- **Auto Rename Tag, Live Server**, un tema a piacere.

Terminale per principianti

Definizione. Il terminale (o shell, o riga di comando) è l'interfaccia testuale del sistema operativo. Scrivi un comando, premi Invio, il computer lo esegue.

Il concetto chiave è che il terminale ha sempre una **posizione corrente** in una cartella. I comandi agiscono lì.

```
pwd          # "print working directory": dove sono adesso
```

```
ls           # elenca i file della cartella corrente (dir su Windows)
```

```
cd Desktop   # entra nella cartella Desktop
```

```
cd ..        # risali di una cartella
```

```
mkdir progetti # crea la cartella "progetti"
```

```
code .       # apre la cartella corrente in VS Code
```

Esempio di flusso reale, quello mostrato nella trascrizione:

```
cd Desktop
```

```
npx create-react-app prima-app
```

```
cd prima-app
```

```
npm start
```

Nota il `cd prima-app`: è l'errore tipico del principiante. Se lanci `npm start` restando sul Desktop, il comando fallisce, perché lì non c'è nessun progetto.

Il corso mostra anche che VS Code ha un **terminale integrato**: `Ctrl + `` su Windows, e ti trovi già posizionato nella cartella del progetto aperto.